12 其他优化领域

在本章中，我们将探讨一些优化主题，它们与前三章中涵盖的任何类别没有特别关联，但仍然足够重要，可以在本书中占有一席之地。

12.1 优化输入输出

待写

- 网络和存储 IO 内核旁路
- 使用特定于操作系统的 api，例如 mmap(将文件读入地址空间) 与 c++ 流。

Architecture-Specific Optimizations

Performance considerations on x86, ARM, and RISC-V

Major differences between ISAs

Know capabilities of your ISA

CISC vs RISC code density

Microarchitecture-specific issues

Memory ordering

Memory alignment

4K aliasing

Cache trashing

Non-temporal stores

Instruction latencies and throughput

12.2 低延迟优化技术

到目前为止，我们已经讨论了各种旨在改进应用程序整体性能的软件优化。在这一节中，我们将讨论在低延迟系统（例如实时处理和高频交易 (HFT)）中使用的额外优化技术。在这样的环境中，主要的优化目标是使程序的特定部分尽可能快地运行。当您在 HFT 行业工作时，每个微秒和纳秒都至关重要，因为它直接影响利润。通常，低延迟部分会实现实时或 HFT 系统的关键循环，例如移动机械臂或向交易所发送订单。优化关键路径的延迟有时会以牺牲程序其他部分为代价。一些技术甚至会牺牲整个系统的整体吞吐量。

当开发人员优化延迟时，他们会避免在热点路径上支付任何不必要的开销。这通常涉及系统调用、内存分配、I/O 以及任何其他具有非确定性延迟的内容。为了达到最低可能的延迟，热点路径需要提前准备好所有资源并可供其使用。

一个相对简单的方法是预先计算一些会在热点路径上执行的操作。这会带来使用更多内存的代价，这些内存将无法供系统中的其他进程使用，但它可以为您在关键路径上节省一些宝贵的周期。但是，请记住，有时计算内容比从内存中获取结果更快。

既然这是一本关于低级 CPU 性能的书籍，我们将跳过讨论类似于刚才提到的更高层次的技术。相反，我们将讨论如何在关键路径上避免页面错误 (page faults)、缓存未命中 (cache misses)、TLB 驱逐 (TLB shootdowns) 和核心节流 (core throttling)。

12.2.1 避免小页面错误

虽然这个术语中包含了“小 (minor)”这个词，但小页面错误对运行时延迟的影响绝对不可小觑。回想一下，当用户代码分配内存时，操作系统只承诺提供一页，但并不会立即通过给我们一个零初始化的物理页面来履行承诺。相反，它会等到用户代码第一次访问它时，然后操作系统才履行其职责。对新分配页面的第一次访问会触发一个小页面错误，这是一个由操作系统处理的硬件中断。小错误对延迟的影响可以从不到一微秒到几微秒不等，特别是如果你使用的是具有 5 级页面表而不是 4 级页面表的 Linux 内核。

你如何在你的应用程序中检测运行时小页面错误？一个简单的方法是使用`top`工具（添加`-H`选项以查看线程级别的视图）。添加`vMn`字段到默认的显示列选择中，以查看每个显示刷新间隔期间发生的小页面错误数量。Listing 12.2.1显示了在编译大型 C++ 项目时，前 10 个进程的`top`命令的输出。额外的`vMn`列显示了在过去 3 秒内发生的小页面错误数量。

代码清单: 编译大型 c++ 项目时，带有附加 vMn 字段的 Linux top 命令的转储。

```
   PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND  vMn
341763 dendiba+  20   0  303332 165396  83200 R  99.3   1.0   0:05.09 c++      13k
341705 dendiba+  20   0  285768 153872  87808 R  99.0   1.0   0:07.18 c++       5k
341719 dendiba+  20   0  313476 176236  83328 R  94.7   1.1   0:06.49 c++       8k
341709 dendiba+  20   0  301088 162800  82944 R  93.4   1.0   0:06.46 c++       2k
341779 dendiba+  20   0  286468 152376  87424 R  92.4   1.0   0:03.08 c++      26k
341769 dendiba+  20   0  293260 155068  83072 R  91.7   1.0   0:03.90 c++      22k
341749 dendiba+  20   0  360664 214328  75904 R  88.1   1.3   0:05.14 c++      18k
341765 dendiba+  20   0  351036 205268  76288 R  87.1   1.3   0:04.75 c++      18k
341771 dendiba+  20   0  341148 194668  75776 R  86.4   1.2   0:03.43 c++      20k
341776 dendiba+  20   0  286496 147460  82432 R  76.2   0.9   0:02.64 c++      25k
```

另一种检测运行时小页面错误的方法是使用 `perf stat -e page-faults` 附加到正在运行的进程。

在 HFT 世界中，任何超过 0 的页错误都是一个问题。但是对于其他业务领域的低延迟应用程序来说，每秒钟出现 100-1000 次错误的持续情况应该进一步调查。调查运行时小页面错误的根本原因可以像启动 `perf record -e page-faults` 然后 `perf report` 一样简单，以定位有问题的源代码行。

为了在运行时避免页面错误惩罚，您应该在启动时预先为应用程序分配所有内存。一个示例代码可能看起来像这样:

```
char *mem = malloc(size);
int pageSize = sysconf(_SC_PAGESIZE)
for (int i = 0; i < size; i += pageSize)
    mem[i] = 0;
```

首先，这段示例代码像往常一样在堆上分配了 `size` 大小的内存。然而，紧接在分配之后，它会逐个页面访问新分配的内存，确保每个页面都加载到 RAM 中。这种方法有助于避免未来访问时因小页面错误造成的运行时延迟。

请看 Listing 201 中更全面的方法，它结合了 mlock/mlockall 系统调用对 glibc 分配器进行调整（摘自“实时 Linux Wiki” [201]）。

代码清单: 调整 glibc 分配器以锁定内存中的页，并防止将它们释放到操作系统。

```
#include <malloc.h>
#include <sys/mman.h>

mallopt(M_MMAP_MAX, 0);
mallopt(M_TRIM_THRESHOLD, -1);
mallopt(M_ARENA_MAX, 1);

mlockall(MCL_CURRENT | MCL_FUTURE);

char *mem = malloc(size);
for (int i = 0; i < size; i += sysconf(_SC_PAGESIZE))
    mem[i] = 0;
//...
free(mem);
```

在 Listing 201中的代码调整了 glibc malloc 的三个设置：M_MMAP_MAX、M_TRIM_THRESHOLD和M_ARENA_MAX。

- 将M_MMAP_MAX设置为0会禁用对于大内存分配的底层mmap系统调用使用——这是必要的，因为当库尝试释放mmap分配的段回操作系统时，mlockall的效果可能会被munmap撤销，这会破坏我们这么做的目的。
- 将M_TRIM_THRESHOLD设置为-1可以防止 glibc 在free调用后将内存返回给操作系统。如前所述，这个选项对mmap分配的段没有影响。
- 最后，将M_ARENA_MAX设置为1可以防止 glibc 通过mmap为多个核心分配多个内存区域。请注意，后者会妨碍 glibc 分配器的多线程可扩展性特性。

综合这些设置，会强制 glibc 进行堆分配，这些分配在应用程序结束前不会将内存释放回操作系统。因此，在上述代码中free(mem)的最终调用后，堆的大小将保持不变。如果初始化时分配的空间足够大，任何后续的运行时调用malloc或new都将简单地重用这个预先分配/预先错误的堆区域中的空间。

更重要的是，由于之前的mlockall调用，for循环中预先错误的所有堆内存将保持在 RAM 中——选项MCL_CURRENT锁定当前映射的所有页面，而MCL_FUTURE锁定将来会成为映射的所有页面。使用mlockall的这种方式还有一个额外的好处，即由此进程生成的任何线程的栈也将被预先错误和锁定。对于更精细的页面锁定控制，开发者应该使用mlock系统调用，它允许你选择哪些页面应该保持在 RAM 中。这种技术的副作用是它会减少系统上其他正在运行的进程可用的内存量。

Windows 应用程序的开发者应该查看以下 API：使用VirtualLock锁定页面，使用带有MEM_DECOMMIT但不带有MEM_RELEASE标志的VirtualFree避免立即释放内存。

这只是防止运行时小页面错误的两种示例方法。这些技术中的一些或全部可能已经集成到如 jemalloc、tcmalloc 或 mimalloc 等内存分配库中。检查你选择的库的文档，看看有哪些可用的选项。

12.2.2 高速缓存预热

在某些应用程序中，延迟最敏感的部分代码是最不经常执行的。例如，高频交易应用程序会持续从证券交易所读取市场数据信号，一旦检测到有利信号，就会向交易所发送买入订单。在上例工作负载中，读取市场数据的代码路径

[201] Linux 基金会 Wiki：实时应用程序内存 - https://wiki.linuxfoundation.org/realtime/documentation/howto/applications/memory

是最常执行的，而执行买入订单的代码路径很少执行。

由于市场上的其他参与者也可能捕捉到相同的市场信号，策略的成功在很大程度上取决于我们的反应速度，换句话说，取决于我们发送订单到交易所的速度。当我们希望我们的买入订单尽快到达交易所并利用市场数据中检测到的有利信号时，我们最不想遇到的是在决定起飞的那一刻遇到障碍。

当一段代码路径一段时间没有运行时，它的指令和相关数据很可能从 I-cache 和 D-cache 中被驱逐出去。然后，就在我们需要运行这段关键的很少执行的代码时，我们遇到了 I-cache 和 D-cache 未命中惩罚，这可能会让我们输掉比赛。这就是缓存预热 (cache warming) 技术可以发挥作用的地方。

缓存预热涉及定期执行延迟敏感的代码以将其保留在缓存中，同时确保它不会执行任何不需要的操作。执行延迟敏感的代码也会通过将延迟敏感的数据带入其中来“预热”D-cache。这种技术通常用于高频交易应用程序。虽然我们不会提供示例实现，但您可以在 CppCon 2018 lightning talk: https://www.youtube.com/watch?v=XzRxikGgaHI[202] 中体验一下。

12.2.3 避免 TLB 驱逐

我们从前面章节了解到，TLB 是每个内核的一个快速但有限的虚拟到物理内存地址转换缓存，它减少了耗时的内核页表遍历的需要。当一个进程从一个内核被调度出去，为一个具有完全不同的虚拟地址空间的新进程让路时，属于该内核的 TLB 需要被刷新。除了批发性的 TLB 刷新之外，还有一个更具选择性的过程来使无效的 TLB 条目称为 TLB 驱逐。

与基于 MESI 的协议和每个内核的 CPU 缓存（即 L1、L2 和 LLC）不同，硬件本身无法维护核心到核心之间的 TLB 一致性。因此，这项任务必须由内核通过软件来完成。内核通过一种特定的处理器间中断 (IPI) 来实现这个角色，叫做 TLB 驱逐，在 x86 平台上通过 `INVLPG` 汇编指令实现。

TLB 驱逐是实现多线程应用程序低延迟时最容易忽视的陷阱之一。为什么？因为在多线程应用程序中，进程线程共享虚拟地址空间。因此，内核必须在参与线程运行的内核的 TLB 之间通信特定类型的对该共享地址空间的更新。例如，常用的系统调用，如 `munmap`（可以禁用 glibc 分配器使用，参见 Section 12.2.1）、`mprotect` 和 `madvise`，会影响内核必须在进程的组成线程之间通信的地址空间更改类型。

尽管开发人员可能避免在他的代码中显式使用这些系统调用，但 TLB 驱逐仍然可能来自外部源 - 例如，分配器共享库或操作系统设施。这种类型的 IPI 不仅会扰乱运行时应用程序性能，而且其影响的程度会随着所涉及线程数量的增加而增大，因为中断是在软件中传递的。

如何检测多线程应用程序中的 TLB 驱逐？一种简单的方法是检查 `/proc/interrupts` 中的 TLB 行。一种检测运行时连续 TLB 中断的有用方法是在查看此文件时使用 `watch` 命令。例如，您可以运行 `watch -n5 -d 'grep TLB /proc/interrupts'`, 其中 `-n 5` 选项每 5 秒刷新视图，而 `-d` 则突出显示每次刷新输出之间的差异。

Listing 12.2.3 显示了在运行延迟关键线程的 `CPU2` 处理器上出现大量 TLB 驱逐的 `/proc/interrupts` 转储。注意其他内核数量级上的差异。在这种情况下，这种行为的罪魁祸首是 Linux 内核的一个名为自动 NUMA 平衡的功能，可以通过 `sysctl -w numa_balancing=0` 轻松禁用。

代码清单: 一个/proc/interrupts 的转储文件，其中显示了 CPU2 上大量 TLB 被击落的情况

```
           CPU0       CPU1       CPU2       CPU3
...
NMI:          0          0          0          0   Non-maskable interrupts
LOC:     552219    1010298    2272333    3179890   Local timer interrupts
SPU:          0          0          0          0   Spurious interrupts
...
```

[202] 缓存预热技术 - https://www.youtube.com/watch?v=XzRxikGgaHI

```
IWI:          0          0          0          0   IRQ work interrupts
RTR:          7          0          0          0   APIC ICR read retries
RES:      18708       9550        771        528   Rescheduling interrupts
CAL:        711        934       1312       1261   Function call interrupts
TLB:       4493       6108      73789       5014   TLB shootdowns
```

但这不是导致 TLB 驱逐的唯一来源。其他来源还包括透明大页、内存压缩、页面迁移和页面缓存回写。垃圾回收器也可以启动 TLB 驱逐。这些特性在履行其职责的过程中会重新定位页面和/或更改页面权限，这需要更新页表，从而导致 TLB 驱逐。

防止 TLB 驱逐需要限制对共享进程地址空间进行的更新次数。在源代码层面，您应该避免运行时执行上述系统调用列表，即 `munmap`、`mprotect` 和 `madvise`。在操作系统层面，禁用内核功能，这些功能会因其功能而导致 TLB 驱逐，例如透明大页和自动 NUMA 平衡。有关 TLB 驱逐的更多细致讨论，以及它们的检测和预防，请阅读 JabPerf 博客上的文章: https://www.jabperf.com/how-to-deter-or-disarm-tlb-shootdowns/[203]。

12.2.4 防止意外内核节流

C/C++ 编译器是工程领域的杰出成就。然而，它们有时会生成令人惊讶的结果，可能让你陷入无谓的追查中。一个现实例子是编译器优化器发出了你从未打算过的繁重的 AVX 指令。虽然在更现代的芯片上这个问题小了很多，但许多较老的 CPU（仍在本地和云端积极使用）在执行繁重的 AVX 指令时会出现严重的内核节流/降频现象。如果你的编译器在没有你的明确知情或同意的情况下产生了这些指令，你可能会在应用程序运行期间遇到无法解释的延迟异常。

针对这种情况，如果您不希望使用繁重的 AVX 指令，可以将 "-mprefer-vector-width=###" 添加到您的编译标志中，将最高宽度指令集固定为 128 或 256。同样，如果您整个服务器集群运行的是最新芯片，那么也不用太担心，因为 AVX 指令集的节流影响现在已经微乎其微。

12.3 缓慢的浮点运算

一些进行大量浮点值计算的应用程序容易出现一个非常微妙的问题，从而导致性能下降。这个问题出现在应用程序遇到 "次规格" 浮点值时，我们将在本节讨论。您还可以找到术语 "去规格" 浮点值，它是指同一个东西。根据 IEEE 标准 754，[204] 次规格值是一个非零数字，其指数小于最小规格数。[205] Listing 205 展示了一个非常简单的次规格值的实例。

在实际应用中，次规格值通常表示一个非常小的信号，以至于它与零无法区分。在音频中，它可能意味着一个非常安静的信号，超出了人类的听觉范围。在图像处理中，它可以表示像素的任何 RGB 颜色分量非常接近于零，等等。有趣的是，次规格值存在于许多生产软件包中，包括天气预报、光线追踪、物理模拟和建模等等。

代码清单: 实例化一个正常和次正常的 FP 值

```
unsigned usub = 0x80200000; // -2.93873587706e-39 (subnormal)
unsigned unorm = 0x411a428e; // 9.641248703 (normal)
float sub = *((float*)&usub);
float norm = *((float*)&unorm);
assert(std::fpclassify(sub) == FP_SUBNORMAL);
assert(std::fpclassify(norm) != FP_SUBNORMAL);
```

[203] JabPerf 博客: TLB 驱逐 - https://www.jabperf.com/how-to-deter-or-disarm-tlb-shootdowns/

[204] IEEE 754 标准 - https://ieeexplore.ieee.org/document/8766229

[205] 次规格数 - https://en.wikipedia.org/wiki/Subnormal_number

如果没有次规格值，两个浮点值 `a - b` 的减法可能会溢出并产生零，即使这两个值不相等。次规格值允许计算逐渐失去精度，而不会将结果舍入为零。不过，正如我们稍后将看到的，这也是有代价的。次规格值也可能出现在生产软件中，当一个值在一个循环中不断减小或除法时出现。

从硬件的角度来看，处理次规格值比处理普通浮点值更困难，因为它需要特殊处理，通常被认为是一种特殊情况。应用程序不会崩溃，但性能会下降。生成或消耗次规格值的计算比对普通数字执行类似计算要慢得多，速度可以慢 10 倍甚至更多。例如，英特尔处理器目前使用微码 协助 处理次规格值操作。当处理器识别到次规格浮点值时，微码执行器 (MSROM) 将提供必要的微操作 (μops) 来计算结果。

在许多情况下，次规格值是由算法自然生成的，因此是不可避免的。幸运的是，大多数处理器都提供了将次规格值刷新为零并一开始不生成次规格值的选项。事实上，许多用户宁愿结果稍微不那么准确，也不愿让代码变慢。不过，对于金融软件来说，相反的论点也可以成立：如果你将次规格值刷新为零，你就失去了精度，并且无法将其向上扩展，因为它仍然是零。这可能会让一些客户生气。

假设你可以接受没有次规格值，那么如何检测和禁用它们？虽然可以使用 Listing 205 所示的运行时检查，但在整个代码库中插入它们并不实际。使用 PMU（性能监控单元）检测应用程序是否生成次规格值是一种更好的方法。在英特尔 CPU 上，你可以收集 `FP_ASSIST.ANY` 性能事件，每次使用次规格值时都会增加该事件。TMA 方法将这种瓶颈归类为 “Retiring” 类别，是的，这是高 “Retiring” 值不好的情况之一。

一旦确认存在次规格值，你可以启用 FTZ 和 DAZ 模式：

- **DAZ** (Denormals Are Zero)。任何低于正常值的输入在使用之前都被替换为零。
- **FTZ** (Flush To Zero)。任何会变为低于正常值的输出都替换为零。

启用它们后，CPU 浮点运算中就不需要昂贵地处理次规格值了。在 x86 平台上，`MXCSR`（全局控制和状态寄存器）中有两个独立的位字段。在 ARM Aarch64 中，两种模式由 `FPCR` 控制寄存器的 `FZ` 和 `AH` 位控制。如果你用 `-ffast-math` 编译你的应用程序，就不用担心了，编译器会自动在程序开始时插入所需代码启用这两个标志。`-ffast-math` 编译器选项有点过载，所以 GCC 开发人员创建了一个单独的 `-mdaz-ftz` 选项，只控制次规格值的行为。如果你想从源代码控制它，Listing 205 显示了你可以使用的示例。如果你选择这个选项，请避免频繁更改 `MXCSR` 寄存器，因为操作相对昂贵。读取 MXCSR 寄存器有相当长的延迟，写入寄存器是一个序列化指令。

代码清单: 手动启用 FTZ 和 DAZ 模式

```
unsigned FTZ = 0x8000;
unsigned DAZ = 0x0040;
unsigned MXCSR = _mm_getcsr();
_mm_setcsr(MXCSR | FTZ | DAZ);
```

请注意，`FTZ` 和 `DAZ` 模式都与 IEEE 754 标准不兼容。它们是在硬件中实现的，以提高在溢出常见且生成非规格化结果不必要时的应用程序性能。通常，我们观察到一些使用次规格值的生产浮点应用程序速度提高了 3% - 5%，有时甚至高达 50%。

12.4 系统调优

在成功完成了利用 CPU 微架构所有复杂设施调整应用程序的所有艰苦工作之后，我们最不想看到的是系统固件、操作系统或内核破坏我们所有的努力。即使是最精细调整的应用程序，如果被系统管理中断 (SMI)（一种 BIOS 中断，用于停止整个操作系统以执行固件代码）间歇性地中断，也毫无意义。这样的中断每次可能会运行 10 到 100 毫秒。

公平地说，开发人员通常对应用程序执行环境几乎没有控制权。当我们交付产品时，不切实际地调整客户可能拥有的每个设置。通常，足够大的组织会有单独的运维团队来处理这类问题。尽管如此，与这些团队成员沟通时，了解还有哪些因素会限制应用程序表现出最佳性能是很重要的。

正如 Section 2.1 所示，现代系统中有许多需要调整的地方，避免系统级干扰并非易事。基于 x86 的服务器部署的性能调优手册的一个例子是 Red Hat 的 指南: https://access.redhat.com/sites/default/files/attachments/201501-perf-brief-low-latency-tuning-rhel7-v2.1.pdf[206]。在那里，您将找到消除或显著减少缓存破坏中断的提示，这些中断来自系统 BIOS、Linux 内核和设备驱动程序等许多应用程序干扰源。这些指南应作为所有新服务器构建的基础映像，然后再将任何应用程序部署到生产环境中。

当涉及调整特定系统设置时，它并不总是简单的“是”或“否”答案。例如，您的应用程序是否会从其所在环境中启用的同步多线程 (SMT) 功能中受益，一开始并不清楚。一般指导原则是仅为 IPC 相对较低的异构工作负载[207] 启用 SMT。另一方面，如今的 CPU 制造商提供具有如此高的内核数量的处理器，以至于 SMT 比过去少得多。然而，这只是一个通用指南，正如本书迄今强调的那样，最好是自己去测量。

大多数开箱即用的平台都配置为在可能的情况下优化吞吐量并节省电量。但是，有一些行业具有实时要求，它们更关心降低延迟而不是其他一切。这样的行业的一个例子是用于汽车装配线的机器人。此类机器人执行的动作由外部事件触发，并且通常有一个预定的时间预算来完成，因为下一个中断即将到来（通常称为“控制环路”）。满足此类平台的实时目标可能需要牺牲机器的整体吞吐量或允许其消耗更多能量。该领域流行的技术之一是禁用处理器睡眠状态[208]，使其时刻准备立即做出反应。另一种有趣的方法称为缓存锁定，[209] 其中 CPU 缓存的部分保留用于特定数据集；它有助于简化应用程序内的内存延迟。

12.5 案例研究：对最后一级缓存大小的敏感性

本案例研究的目的是展示如何确定应用程序是否对最后一级缓存 (LLC) 的大小敏感。利用这些信息，您可以在购买计算系统硬件组件时做出明智的决策。同样，您以后可以确定对其他因素（例如内存带宽、核心数量、处理器频率等）的敏感性，并可能购买更便宜的计算机，同时保持相同的性能水平。

在这个案例研究中，我们对同一组应用程序运行多次，LLC 大小各不相同。现代服务器处理器允许用户控制 LLC 空间分配给处理器线程。通过这种方式，用户可以限制每个线程仅使用其分配的共享资源数量。此类设施通常称为服务质量 (QoS) 扩展。它们可用于优先处理关键性能应用程序并减少与同一系统中其他线程的干扰。除了 LLC 分配之外，QoS 扩展还支持限制内存读取带宽。

我们的分析将帮助我们识别当 LLC 大小减小时性能显著下降的应用程序。我们称这样的应用程序对 LLC 大小敏感。我们还确定了不敏感的应用程序，即 LLC 大小不会影响性能。此结果可用于正确调整处理器 LLC 的大小，特别是考虑到市场上可用范围广泛。例如，我们可以确定应用程序是否可以从更大的 LLC 中受益，即投资新硬件是否合理。相反，如果应用程序对小缓存大小已足够，那么我们可以购买更便宜的处理器。

对于本案例研究，我们使用 AMD Milan 处理器，但其他服务器处理器，例如 Intel Xeon 和 ARM ThunderX，也包含硬件支持，允许用户控制 LLC 空间和内存读取带宽分配给处理器线程。

目标机器：AMD EPYC 7313P

我们使用了一个服务器系统，该系统配有 16 核 AMD EPYC 7313P 处理器，代号为 Milan，AMD 于 2021 年推出。该系统的关键特性在表 10 中指定。

[206] Red Hat 低延迟调优指南 - https://access.redhat.com/sites/default/files/attachments/201501-perf-brief-low-latency-tuning-rhel7-v2.1.pdf

[207] 即，当兄弟线程执行不同的指令模式时

[208] 电源管理状态：P 状态、C 状态。详细信息请参见此处：https://software.intel.com/content/www/us/en/develop/articles/power-management-states-p-states-c-states-and-package-c-states.html.

[209] 缓存锁定。缓存锁定技术的调查 [Mittal, 2016]。将缓存的一部分伪锁定的示例，然后将其作为 Linux 文件系统中的字符设备公开并可用于 mmaping：https://events19.linuxfoundation.org/wp-content/uploads/2017/11/Introducing-Cache-Pseudo-Locking-to-Reduce-Memory-Access-Latency-Reinette-Chatre-Intel.pdf.

Table 10: 实验所用服务器的主要特性。

特性	值
处理器 Processor	AMD EPYC 7313P
核心数 x 线程数 Cores x threads	16 × 2
配置 Configuration	4 CCX × 4 个核心/CCX
频率 Frequency	3.0/3.7 GHz, 基础/最大
L1 缓存 (I, D) L1 cache (I, D)	8 通道，32 KiB (每个核心)
L2 缓存 L2 cache	8 通道，512 KiB (每个核心)
LLC	16 通道，32 MiB，非包容式 (每个 CCX) 16-ways, 32 MiB, non-inclusive (per CCX)
主内存 Main Memory	512 GiB DDR4，8 个通道，标称峰值带宽：204.8 GB/s 512 GiB DDR4, 8 channels, nominal peak BW: 204.8 GB/s,
TurboBoost TurboBoost	已禁用 Disabled
超线程 Hyperthreading	已禁用 (1 个线程/核心) Disabled (1 thread/core)
操作系统 OS	Ubuntu 22.04，内核 5.15.0-76 Ubuntu 22.04, kernel 5.15.0-76

12.6 AMD Milan 7313P 处理器的集群式内存层次结构

图 73 展示了 AMD Milan 7313P 处理器的集群式内存层次结构。它由四个核心复合芯片 (CCD) 组成，这些 CCD 通过一个 I/O 芯片连接彼此和片外内存。每个 CCD 集成一个核心复合体 (CCX) 和一个 I/O 连接。反过来，每个 CCX 都有四个 Zen3 内核，可以运行八个线程，共享一个 32 MiB 的受害者 LLC，即 LLC 填充了从 CCX 的四个 L2 缓存驱逐的缓存行。

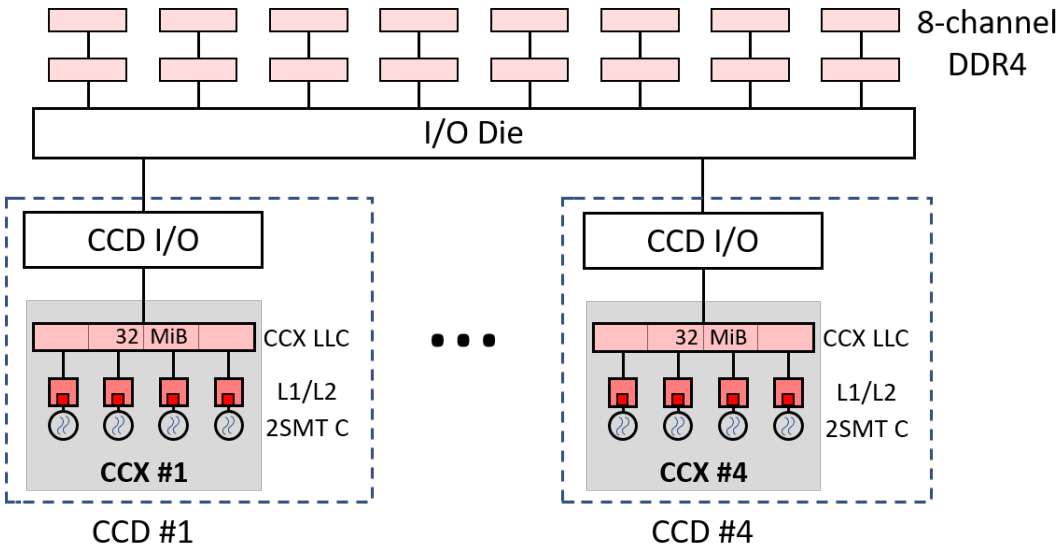


Figure 73: AMD Milan 7313P 处理器的集群式内存层次结构

虽然总共有 128 MiB 的 LLC，但 CCX 的四个内核无法将缓存行存储在除其自己的 32 MiB LLC (32 MiB/CCX x 4 CCX) 以外的 LLC 中。由于我们将运行单线程基准测试，因此我们可以关注单个 CCX。实验中 LLC 的大小将在 0 到 32 MiB 之间变化，步长为 2 MiB。

工作负载：SPEC CPU2017

我们使用 SPEC CPU2017 套件中的部分基准测试。[210] SPEC CPU2017 包含一组行业标准性能基准测试，可对处理器、内存子系统和编译器进行压力测试。它被广泛用于比较高性能系统的性能。它也广泛用于计算机架构研究。

具体来说，我们从 SPEC CPU2017 中选择了 15 个内存密集型基准测试（6 个 INT 和 9 个 FP），正如 [Navarro-Torres et al., 2019] 中建议的那样。这些应用程序使用 GCC 6.3.1 和以下编译器选项编译：`-g -O3 -march=native -fno-unsafe-math-optimizations -fno-tree-loop-vectorize`，正如 SPEC 在套件提供的优化分析文件中指定的。

12.6.1 控制和监控 LLC 分配

为了监控和执行 LLC 分配和内存读取带宽的限制，我们将使用 AMD64 技术平台服务质量扩展 [Advanced Micro Devices, 2022]。用户可以通过特定于模型的寄存器 (MSR) 的库来管理此 QoS 扩展。首先，必须通过写入 `PQR_ASSOC` 寄存器 (MSR `0xC8F`) 为线程或一组线程分配资源管理标识符 (RMID) 和服务类别 (COS)。以下是硬件线程 1 的示例命令：

```
# 写入 PQR_ASSOC (MSR 0xC8F): RMID=1, COS=2 -> (COS << 32) + RMID
$ wrmsr -p 1 0xC8F 0x200000001
```

其中 `-p 1` 表示硬件线程 1. 我们显示的所有 `rdmsr` 和 `wrmsr` 命令都需要 root 访问权限。

LLC 空间管理通过写入每个线程的 16 位二进制掩码执行。掩码的每个位允许线程使用给定的十六分之一的 LLC（在 AMD Milan 7313P 的情况下为 1/16 = 2 MiB）。多个线程可以使用相同的片段，这意味着竞争性地共享相同的 LLC 子集。

要设置线程 1 对 LLC 使用的限制，我们需要写入 `L3_MASK_n` 寄存器，其中 `n` 是 COS，即相应 COS 可以使用的缓存分区。例如，要限制线程 1 仅使用 LLC 可用空间的一半，请运行以下命令：

```
# 写入 L3_MASK_2 (MSR 0xC92): 0x00FF (LLC 空间的一半)
$ wrmsr -p 1 0xC92 0x00FF
```

现在，要监控硬件线程 1 对 LLC 的使用情况，首先我们需要将监控标识符 RMID 与 LLC 监控事件 (L3 缓存占用率监控，`evtID 0x1`) 关联起来。我们通过写入 `QM_EVTSEL` 控制寄存器 (MSR `0xC8D`) 来做到这一点。之后，我们应该读取 `QM_CTR` 寄存器 (MSR `0xC8E`)：

```
# 写入 QM_EVTSEL (MSR 0xC8D): RMID=1, evtID=1 -> (RMID << 32) + evtID
$ wrmsr -p 1 0xC8D 0x100000001
# 读 QM_CTR (MSR 0xC8E)
$ rdmsr -p 1 0xC8E
```

这将使我们能够估计缓存行中的 LLC 使用情况[211]。要将此值转换为字节，我们需要将 `rdmsr` 命令返回的值乘以缓存行大小。

同样，可以限制分配给线程的内存读取带宽。这是通过将无符号整数写入特定的 MSR 寄存器来实现的，该寄存器以 1/8 GB/s 的增量设置最大读取带宽。欢迎感兴趣的读者阅读 [Advanced Micro Devices, 2022] 了解更多详细信息。

评估指标

用于量化应用程序性能的最终指标是执行时间。为了分析内存层次结构对系统性能的影响，我们还将使用以下三个指标：1) CPI，每条指令的周期数[212]，2) DMPKI，每千条指令的 LLC 中的需求缺失，以及 3) MPKI，每千条指令的

[210] SPEC CPU® 2017 - https://www.spec.org/cpu2017/

[211] AMD 文档 [Advanced Micro Devices, 2022] 更准确地使用了术语 L3 缓存转换因子，可以通过 `cpuid` 指令确定。

[212] 我们使用 CPI 而不是每条指令的时间，因为我们假设 CPU 频率在实验过程中不会改变。

LLC 中的总缺失（需求 + 预取）。虽然 CPI 与应用程序性能直接相关，但 DMPKI 和 MPKI 不一定会影响性能。表 11 显示了用于从特定硬件计数器计算每个指标的公式。有关每个计数器的详细描述，请参见 AMD 的处理器编程参考 [Advanced Micro Devices, 2021]。

Table 11: 案例研究中使用的指标计算公式。

指标	公式
CPI	非暂停周期 (PMCx076) / 退役 (retired) 指令 (PMCx0C0) Cycles not in Halt (PMCx076) / Retired Instructions (PMCx0C0)
DMPKI	需求数据缓存填充[213] (PMCx043) / (退役 (retired) 指令 (PMCx0C0) / 1000) Demand Data Cache Fills[214] (PMCx043) / (Retired Instructions (PMCx0C0) / 1000)
MPKI	L3 缺失[215] (L3PMCx04) / (退役 (retired) 指令 (PMCx0C0) / 1000) L3 Misses[216] (L3PMCx04) / (Retired Instructions (PMCx0C0) / 1000)

硬件计数器可以通过 MSR 进行配置和读取。配置包括指定要监视的事件以及如何监视它。在我们的系统中，每个线程有 6 个核心计数器，每个 L3-CCX 有 6 个计数器，以及 4 个数据结构计数器。访问核心事件通过写入 `PERF_CTL[0-5]` 控制寄存器 (MSR `0xC001020[0,2,4,6,8,A]`) 完成。`PERF_CTR[0-5]` 寄存器 (MSR `0xC001020[1,3,5,7,9,B]`) 是与这些控制寄存器关联的计数器。例如，对于计数器 0 来收集在硬件线程 1 上运行的应用程序退役 (retired) 的指令数，执行以下命令：

```
$ wrmsr -p 1 0xC0010200 0x5100C0
$ rdmsr -p 1 0xC0010201
```

其中 `-p 1` 表示硬件线程 1，`0xC0010200` 是计数器 0 (`PERF_CTL[0]`) 的控制 MSR，`0x5100C0` 指定要测量的事件标识符（已退役 (retired) 指令，`PMCx0C0`）以及如何测量它（用户事件）。使用 `wrmsr` 完成配置后，可以使用 `rdmsr` 命令读取收集已退役 (retired) 指令数的计数器 0。

类似地，通过写入 L3 控制寄存器 (MSR `0xC001023[0,2,4,6,8,A]`) 并读取其关联计数器 (MSR `0xC001023[1,3,5,7,9,B]`) 来访问 L3 缓存事件。最后，通过写入 `DF_PERF_CTL[0-3]` 控制寄存器 (MSR `0xC001024[0,2,4,6]`) 并读取其关联的 `DF_PERF_CTR[0-3]` 寄存器 (MSR `0xC001024[1,3,5,7]`) 来访问数据结构事件[217]。

本案例研究中使用的的方法在 [Navarro-Torres et al., 2023] 中更详细地描述。可以在以下公共存储库中找到重现实验所需的代码和信息：https://github.com/agusnt/BALANCER。

结果

我们在系统中单独运行一套 SPEC CPU2017 基准测试，仅使用一个实例和单个硬件线程。重复这些运行，同时将可用 LLC 大小从 0 更改为 32 MiB，步长为 2 MiB。图 74 从左到右以图形方式显示每个分配的 LLC 大小的 CPI、DMPKI 和 MPKI。对于 CPI 图表，Y 轴上的较低值意味着更好的性能。此外，由于系统上的频率是固定的，因此 CPI 图表反映了绝对分数。例如，具有 32 MiB LLC 的 `520.omnetpp` (虚线) 比 0 MiB LLC 快 2.5 倍。

对于 DMPKI 和 MPKI 图表，Y 轴上的值越低越好。对应于 `503.bwaves` (实线)、`520.omnetpp` (虚线) 和 `554.roms` (虚线) 的三条线代表了所有应用程序中观察到的三个主要趋势。我们不显示其余基准测试。

[213] 我们使用了`MemIoRemote`和`MemIoLocal`这两个子变量，它们从连接在远程/本地 NUMA 节点上的 DRAM 或 IO 请求填充数据缓存。
[214] 我们使用了`MemIoRemote`和`MemIoLocal`这两个子变量，它们从连接在远程/本地 NUMA 节点上的 DRAM 或 IO 请求填充数据缓存。
[215] 我们仅使用掩码计算 L3 缺失，具体为 `L3Event[0x0300C00000400104]`。
[216] 我们仅使用掩码计算 L3 缺失，具体为 `L3Event[0x0300C00000400104]`。
[217] 在我们的研究中，我们没有使用数据结构计数器。

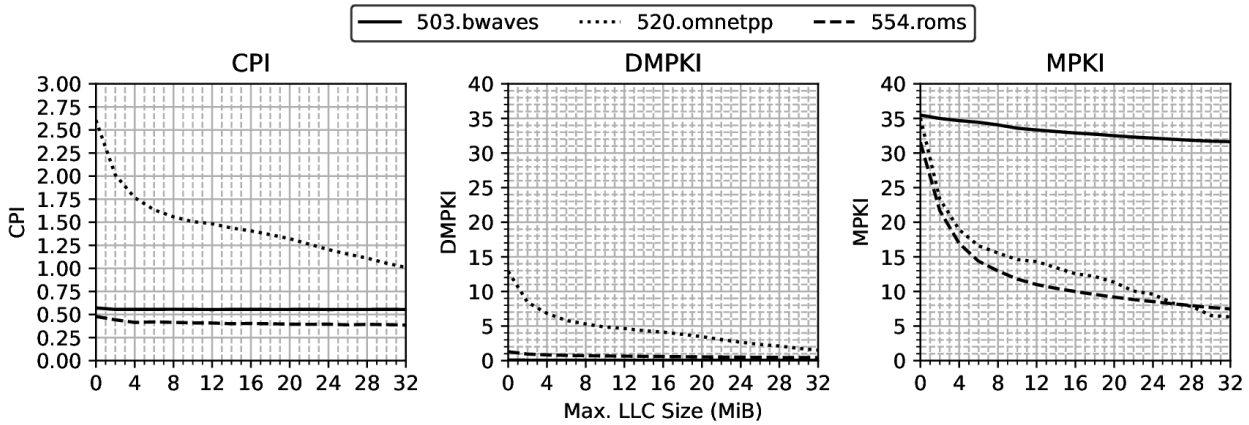


Figure 74: 随着 LLC 分配限制增加 (2 MiB 步长)，CPI, DMPKI 和 MPKI.

在 CPI 和 DMPKI 图表中可以区分两种行为。一方面，520.omnetpp 利用了其在 LLC 中的可用空间：随着 LLC 中分配的空间增加，CPI 和 DMPKI 都显着降低。我们可以说 520.omnetpp 的行为对 LLC 可用大小敏感。增加分配的 LLC 空间可以提高性能，因为它避免了驱逐将来会使用的缓存行。

相比之下，503.bwaves 和 554.roms 不会利用所有可用的 LLC 空间。对于这两个基准测试，随着 LLC 中的分配限制增加，CPI 和 DMPKI 大致保持不变。我们可以说这两个应用程序的性能对其在 LLC 中的可用空间不敏感。如果我们的应用程序表现出这种行为，我们可以选择具有较小 LLC 尺寸的处理器而不牺牲性能。

现在让我们分析 MPKI 图表，其中结合了 LLC 需求缺失和预取请求。首先，我们可以看到 MPKI 值总是远高于 DMPKI 值。也就是说，大多数块都是由预取器从内存加载到芯片内层次结构中的。这种行为是由于预取器在预加载私有缓存以供使用的数据方面非常高效，从而消除了大多数需求缺失。

对于 503.bwaves，我们观察到 MPKI 与 CPI 和 DMPKI 图表大致保持在相同水平。基准测试中可能没有太多数据重用和/或内存流量非常低。520.omnetpp 工作负载的行为与我们之前确定的相同：MPKI 随着可用空间的增加而减少。但是对于 554.roms，MPKI 图表显示随着可用空间增加，总缺失急剧下降，而 CPI 和 DMPKI 保持不变。在这种情况下，基准测试中存在数据重用，但这对性能无关紧要。预取器可以提前获取所需数据，消除需求缺失，无论 LLC 中可用空间如何。但是，随着可用空间减少，预取器无法在 LLC 中找到块并且必须从内存加载它们的可能性会增加。因此，为这类应用程序提供更多 LLC 容量不会直接提升其性能，但会使系统受益，因为它减少了内存流量。

通过查看 CPI 和 DMPKI，我们最初认为 554.roms 对 LLC 大小不敏感。但通过分析 MPKI 图表，我们需要重新考虑我们的陈述并得出结论，即 554.roms 也对 LLC 大小敏感，因此最好不要限制其可用 LLC 空间，以免增加内存带宽消耗。更高的带宽消耗可能会增加内存访问延迟，进而意味着系统上运行的其他应用程序的性能下降 [Navarro-Torres et al., 2023]。

问题和练习

1. 完成 perf-ninja::lto 和 perf-ninja::io_opt1 实验作业。
2. 运行您日常使用的应用程序。找到热点。检查它是否可以从本章讨论的任何技术中受益。

AI 回答：

1. 关于完成 perf-ninja::lto 和 perf-ninja::io_opt1 实验作业的问题，这些作业通常是指在性能优化和分析领域中的实践任务。perf-ninja 是一个性能分析工具，它可以帮助开发者识别程序中的性能瓶颈。lto（Link Time Optimization）是编译器的一个优化选项，它可以在链接时进行跨多个编译单元的优化。io_opt1 可能是指针对输入输出优化的实验。要完成这些实验作业，你需要：

- 使用 perf-ninja 或类似的性能分析工具来收集你的应用程序的性能数据。
- 分析这些数据以识别性能瓶颈或热点。
- 应用 LTO 优化来减少代码大小和提高执行效率。
- 实施输入输出优化技术，比如减少 I/O 操作、使用异步 I/O、优化数据结构等。
- 重新测试你的应用程序，以验证优化是否有效。

请注意，这些步骤需要你有一定的编程和性能分析知识，以及对所使用的工具和编译器的了解。

2. 对于运行日常使用的应用程序并找到热点的问题，你可以采取以下步骤：

- 使用性能分析工具（如 perf、gprof 或 VisualVM 等）来监控你的应用程序的运行。
- 识别程序中的热点，即那些占用大量 CPU 时间或 I/O 资源的部分。
- 根据本章讨论的技术，检查这些热点是否可以从特定的优化中受益。例如，如果热点是由于频繁的 I/O 操作，你可以尝试使用异步 I/O 或优化数据缓存。如果热点是由于 CPU 密集型计算，你可以考虑使用 LTO 优化或向量化指令集。
- 实施优化措施，并再次运行性能分析来评估优化效果。

这个过程可能需要多次迭代，因为优化通常需要根据实际的性能数据来调整。此外，优化一个方面可能会影响其他方面，所以需要仔细监控和测试以确保整体性能的提升。

章节总结